OpenZeppelin | security

Lombard Strategies Audit



June 26, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 4

Scope 5

System Overview 6

Security Model and Trust Assumptions 8

Critical Severity 13

C-01 Migration Hardcodes Initial Price-Per-Share, Detaching Accrued Yield From Existing Holders 13

Medium Severity 14

M-01 Manager Role Can Bypass Admin-Only Converter Updates by Re-Listing a Deposit Asset 14

M-02 Fee Share Minting Inflates Reported NAV Because pricePerShare Is Not Adjusted 15

M-03 Stale pricePerShare Lacks On-Chain Staleness Bound 17

M-04 Missing Staleness Check When Using Staked LBTC Oracle Ratio 18

M-05 Redemption Requests Lock Payout at Request-Time pricePerShare, Enabling Front-Running of PPS Updates 19

Low Severity 20

L-01 Migration Accepts Decimals Parameter Without Cross-Checking the Underlying Asset 20

L-02 Migration Function Lacks Idempotency Guard, Allowing Silent Accounting State Reset on Repeated Invocation 20

L-03 Zero-Asset Redemption Requests Permanently Trap Burned Shares 21

L-04 postPricePerShare Cannot Update Price When Deposits or Redemptions Are Paused 22

L-05 Dead Code in StrategyBaseUpgradeable 22

L-06 convertToAssets Uses Non-Standard Rounding Direction 23

L-07 Management Fee Accrues While Protocol Is Paused 23

L-08 Management Fee Compounds On Treasury Share Balance 24

L-09 maxDeposit and maxMint Ignore ERC-20 Transfer Pause 24

L-10 Merkle Allowlist Constraints With Zero expected Match Against Truncated Calldata 25

L-11 Merkle Calldata Constraints Do Not Model ABI Decoding for Dynamic Arguments 26

L-12 setManagementFeeBps Retroactively Applies New Rate To Unharvested Interval 26

Notes & Additional Information 27

N-01 Blocklist Enforcement Gaps Allow Sanctioned Value to Exit Through Redemption 27

N-02 ShardBaseUpgradeable NatSpec Misrepresents Validator as Immutable 28

N-03 Missing Security Contact Information 28

N-04 Unused Errors 29

N-05 Unused Imports 29

N-06 Constant Not Using UPPER_CASE Format 30

N-07 Variables Initialized With Their Default Values 30

N-08 Inconsistent CapWords Style 30
N-09 Unused Named Return Variables 31
N-10 Missing @custom:storage-location Annotation on StakedLBTCOracleStorage Struct 32
N-11 Merkle Allowlist Proofs Remain Valid Across Upgrades At Allowlisted Targets 32

Client Reported 33
CR-01 NAV Inflation From Supply Changes Between Snapshot And Price Update Allows Over-Claims Against Real Backing 33

Conclusion 35

Appendix 37
Issue Classification 37

Summary

Type	DeFi
Timeline	From 2026-05-11 To 2026-05-22
Languages	Solidity

Total Issues	30 (22 resolved)
Critical Severity Issues	1 (1 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	5 (2 resolved)
Low Severity Issues	12 (8 resolved)
Notes & Additional Information	11 (10 resolved)
Client Reported Issues	1 (1 resolved)

Scope

OpenZeppelin performed an audit of the lombard-finance/smart-contracts repository at commit `a156835`.

Two files were reviewed as diffs against earlier audited revisions rather than in full:

- `BaseLBTC.sol` was reviewed as the diff from commit `95b424e` against `a156835`.
- `StakedLBTCOracle.sol` was reviewed as the diff from commit `9e5c6b1` against `a156835`.
- Before the start of the audit the Lombard team shared PR 442 which contained a different solution to the `postPricePerShare` function. This subsequent revision of was additionally reviewed.

In scope were the following files:

```
contracts
    ├── LBTC
    │   ├── BaseLBTC.sol (Diff)
    │   └── StakedLBTCOracle.sol (Diff)
    ├── stakeAndBake
    │   └── depositor
    │       └── strategy
    │           └── MultiAssetDepositStrategyDepositor.sol
    └── strategy
        ├── converters
        │   ├── ChainlinkConverter.sol
        │   ├── DirectConverter.sol
        │   └── StakedLBTCConverter.sol
        ├── libraries
        │   └── PriceRateLimiter.sol
        ├── shards
        │   ├── RemoteShard.sol
        │   ├── Shard.sol
        │   └── ShardBaseUpgradeable.sol
        ├── validators
        │   └── MerkleAllowlistValidator.sol
        ├── AsyncRedemptionUpgradeable.sol
        ├── BlocklistOracle.sol
        ├── FeeManagerUpgradeable.sol
        ├── MAWARStrategyMigrated.sol
        ├── MultiAssetDepositUpgradeable.sol
        ├── MultiAssetWithAsyncRedemptionStrategy.sol
        └── StrategyBaseUpgradeable.sol
```

System Overview

The in-scope contracts extend the Lombard Finance protocol with a yield-bearing strategy share token built around LBTC and other Bitcoin-denominated assets. The core component is an upgradeable ERC-20 share token whose unit value tracks an off-chain Net Asset Value (NAV) posted on chain. Users deposit listed ERC-20 assets to mint shares, optionally route those deposits into operational sub-accounts known as shards, and exit through an asynchronous redemption queue. The system also includes a compliance oracle, a stakedLBTC ratio oracle, validator-gated shard execution, and an adapter that lets the existing StakeAndBake flow deposit directly into the strategy.

Strategy share token: `StrategyBaseUpgradeable` defines the share token, the registry of authorized shards, and the posted `pricePerShare` used to convert between assets and shares. It is composed with `FeeManagerUpgradeable` (management and performance fees), `MultiAssetDepositUpgradeable` (multi-asset deposit and mint), and `AsyncRedemptionUpgradeable` (redemption request queue) into the concrete `MultiAssetWithAsyncRedemptionStrategy`. Storage is partitioned per module using ERC-7201 namespaced slots, with no reserved gap arrays. `MAWARStrategyMigrated` is a one-shot variant that initializes the same storage from a legacy `ERC4626VaultWrapper` deployment during an upgrade and is intended to be replaced by the plain implementation after migration.

Pricing and NAV: `pricePerShare` is reported by an off-chain rate setter through `postPricePerShare`. Updates are bounded by a token-bucket rate limiter (`PriceRateLimiter`) expressed in micro-basis-points relative to the current value. When a proposed move exceeds remaining capacity, the setter chooses between reverting and a soft-overflow path that drains the bucket, posts the new price, and pauses deposits and redemption requests. Conversions between deposit assets and the strategy base unit go through `IAssetConverter` adapters: `DirectConverter` is the identity mapping, `ChainlinkConverter` reads a Chainlink aggregator with staleness and L2 sequencer uptime checks, and `StakedLBTCConverter` uses the `ratio()` exposed by `StakedLBTCOracle` to value stakedLBTC against BTC accounting units. The sequencer-uptime check reverts when the configured feed reports the sequencer as down or within the grace period (including a zero `startedAt`); mainnet or sequencer-less deployments are expected to pass the zero address to disable the check, and deployments must select the correct configuration per chain.

Deposits and redemptions: Multi-asset deposit and mint paths pull the deposit token, split off a per-asset fee in basis points to the strategy fee treasury, convert the net amount to shares at the current NAV, mint shares to the receiver, and optionally forward the net deposit tokens to a configured default shard. Redemptions are asynchronous: `requestRedeem` burns the net shares (after an optional fee taken in shares) and records a redeem request denominated in underlying assets at the snapshot price; a role-gated fulfillment function later transfers the underlying directly to each request owner. Redemption fulfillment is intentionally not blocked by the redeem pause, while the price post path requires the strategy to be unpaused.

Shards and `exec`: Shards are upgradeable `BeaconProxy` instances that hold operational liquidity on behalf of the strategy. `Shard` is bound to a single `IStrategy` and exposes `pullFromStrategy` (which calls `requestFunding` on the strategy) and `pushToStrategy` for moving any ERC-20 token between strategy and shard; both are gated by a dedicated transfer role and bypass the validator. `RemoteShard` omits this liquidity pipe and is intended for chains where outbound asset movement is performed solely through validator-gated calls. Both shards expose two `exec` overloads sharing `ShardBaseUpgradeable`: a validator-gated overload that consults an `IShardExecValidator` and a privileged overload that bypasses the validator and is reserved for a dedicated executor role. The reference validator, `MerkleAllowlistValidator`, stores a Merkle root and authorizes calls by verifying a canonical rule leaf together with a calldata constraint match. Each rule is bound to the validator deployment via a domain separator that includes `block.chainid` and `address(this)`.

Compliance, ancillary tokens, and integrations: `BlocklistOracle` maintains a local blocklist and aggregates one or more external sanction lists; its `check` function reverts on hit and is invoked from deposit and redemption paths. A non-zero blocklist oracle is required at initialization and cannot be set to zero by upgrades. `BaseLBTC` is an abstract base used by LBTC ERC-20 deployments that adds a mint and burn pause distinct from the standard transfer pause, plus an EIP-712 fee approval helper. `StakedLBTCOracle` accepts ratio updates carried by a payload signed by a notary consortium; updates are constrained by an interval ahead of `block.timestamp` and by a per-interval ratio threshold, and the ratio is consumed by `StakedLBTCConverter`. Finally, `MultiAssetDepositStrategyDepositor` adapts the existing StakeAndBake flow to deposit into the multi-asset strategy by approving and calling `deposit` on behalf of an authorized caller.

Changes in diff-scoped files: `BaseLBTC` was reviewed only for the changes between its previously audited revision and the current commit. The diff drops an unused signature-utility import, introduces a `MANAGER_ROLE` constant, and reworks the mint and burn pause toggles so that `pauseMintBurn` now requires the guard to be off (via the `whenMintBurnAllowed`

modifier) and `unpauseMintBurn` reverts with a new `MintBurnExpectedPaused` error when called on an unpaused contract instead of silently returning. The right to clear the guard moves from `DEFAULT_ADMIN_ROLE` to the new `MANAGER_ROLE`. `StakedLBTCOracle` was likewise reviewed against its previously audited revision. The current commit makes the contract additionally implement `AggregatorV3Interface`, exposing `latestRoundData` and `decimals` so the ratio can be consumed by integrations expecting a Chainlink-shaped feed. A configurable `ratioThreshold` (defaulting to 0.1%) bounds the magnitude of each ratio update scaled by the time interval between switches, and a new `_validateRatio` helper additionally checks that the payload's `denomHash` matches the configured token and that the switch time is strictly greater than the previous one and within the maximum lookahead. Initialization is now routed through a dedicated `_initRatio` that rejects re-initialization, and `_setNewRatio` rejects a zero ratio. A `prevSwitchTime` field is added to storage so that the historical timestamp can be reported alongside `prevRatio`.

Security Model and Trust Assumptions

The strategy uses `AccessControlDefaultAdminRulesUpgradeable`, which applies a configured delay to default admin handover. Where the codebase delegates capabilities to off-chain components (NAV posting, ratio updates, compliance lists, Merkle allowlist authoring), that delegation is described below as a trust requirement rather than a code-level guarantee. Across all of the privileged roles described below, the implicit baseline is that the holders act in the interest of shareholders: they do not collude, do not configure parameters adversarially, and fulfil their operational duties (NAV posting, redemption fulfillment, allowlist rotation, compliance updates) diligently and within the cadence the protocol assumes. The strategy's pricing and compliance paths additionally rely on several oracle-shaped inputs operating as intended: the off-chain NAV reported through `postPricePerShare`, the `StakedLBTCOracle` ratio fed by the notary consortium and consumed by `StakedLBTCConverter` (and by integrators reading the `AggregatorV3Interface` view), the Chainlink aggregators read by `ChainlinkConverter` together with their sequencer-uptime feeds, and the local and aggregated lists exposed by `BlocklistOracle`.

Default Admin (`DEFAULT_ADMIN_ROLE`): Holds the highest authority across the strategy, shard, blocklist, and validator deployments. Can grant and revoke other roles, register shards, list and unlist deposit assets, replace asset converters, replace the blocklist oracle, replace

shard validators, set the redeem fee, set management and performance fee rates, set the management fee treasury during configuration, and unpause ERC-20 transfers. Default admin handover is subject to the rules delay configured at initialization. Users must trust this role to set non-malicious converter, oracle, and validator addresses, to schedule admin transfers responsibly, and to refrain from listing or relisting assets in ways that would alter accounting against existing shareholders.

Manager (`MANAGER_ROLE`): May set the fee treasury, set per-asset deposit fee basis points, set the price-per-share rate limiter configuration, deregister shards, set the default shard, and unpause deposit, redeem, and ERC-20 transfer paths. Users must trust this role to configure the rate limiter conservatively and to not weaken fee, treasury, or shard parameters in ways that disadvantage existing holders. Spec documentation indicates that the rate limiter is sized by `MANAGER_ROLE` and acted on by a different role, with the intent of separating duties.

Price Setter (`PRICE_SETTER_ROLE`): Submits new `pricePerShare` values through `postPricePerShare`. This is the principal NAV input for the strategy; deposit, mint, redemption pricing, and performance fee accrual all derive from it. Users must trust this role to report accurate prices reflecting the real value of off-strategy holdings, and to choose the soft-overflow flag responsibly when posting a move that would exceed the bucket. The rate limiter caps the magnitude of a single move, but a price setter that posts repeatedly within the refill envelope can still drive cumulative changes over time.

Strategy Pauser and Transfers Pauser (`STRATEGY_PAUSER_ROLE`, `TRANSFERS_PAUSER_ROLE`): Can independently pause deposit, redemption request, and ERC-20 transfer paths. Redemption fulfillment is not gated by the strategy pause, so paying out previously requested redemptions can continue while new flows are blocked. Users must trust pauser roles not to grief legitimate flows, while accepting that pause capability is intended as an incident-response control.

Redemption Payer (`PAY_REDEMPTIONS_ROLE`): Fulfils queued redemption requests in batches, transferring the underlying asset directly from the strategy balance to each request owner. `totalAssets()` is derived from `totalSupply()` and `pricePerShare` rather than from raw ERC-20 balances, but fulfillment transfers the strategy's actual underlying balance, so it requires the strategy to hold sufficient underlying at call time; when off-strategy custody backs the NAV, shard operators are expected to push liquidity back via `pushToStrategy` before fulfillment. There is no on-chain deadline on a request, so users rely on this role to process queued redemptions diligently, to preserve request ordering, and to coordinate with shard operators on liquidity.

Fee-related roles (`PERF_FEE_CLAIM_ROLE`, performance fee setter): The performance fee is harvested by the claim role against a stored high-water mark, and the setter on `DEFAULT_ADMIN_ROLE` may switch rates and elect to forfeit accrued fees. The management fee accrues on every operation that triggers `_accrueManagementFee` and is minted to the management fee treasury. Users must trust the management fee treasury address, and accept dilution implied by the configured rates.

Shard Transferer (`SHARD_TRANSFER_ROLE`): On a `Shard`, may freely move any ERC-20 between the strategy and the shard through `pullFromStrategy` and `pushToStrategy`, both of which bypass the shard validator. The strategy's `requestFunding` accepts any token except the share token itself, so this role can withdraw arbitrary listed and unlisted ERC-20 holdings from the strategy. Users must trust shard transferer accounts to use the pipe only for operational rebalancing rather than to drain assets, since this path provides no policy enforcement.

Privileged Executor (`PRIVILEGED_EXECUTOR_ROLE`): Calls the no-`validatorArgs` overload of `exec` on a shard, which forwards an arbitrary external call without consulting the validator. The call is still bounded by reentrancy guards and the contract's ETH balance, but no policy is enforced on `(to, value, data)`. Users must trust this role to be assigned only to accounts whose external calls do not need validator review, since holding this role is equivalent to operating the shard outside the allowlist.

Shard Validator owner (`MerkleAllowlistValidator`): The validator uses `Ownable2Step`; the owner sets and rotates the Merkle root authorizing shard `exec` calls. The leaf encoding binds each rule to `block.chainid` and the validator's own address, so the off-chain tree must be rebuilt for any redeployment or chain. Users must trust the owner to author rules that are neither overly permissive (for example, leaving selectors, recipients, or calldata constraints unrestricted beyond what operations require) nor mutually conflicting in ways that would authorize calls outside intended policy, and to rotate roots promptly when allowed actions change.

Blocklist Setter (`BLOCKLIST_SETTER_ROLE` on `BlocklistOracle`): Adds accounts to the local blocklist; removals require `DEFAULT_ADMIN_ROLE`. The oracle additionally aggregates external sanction lists added by the default admin. Deposits and redemptions revert when `check` denies the sender or receiver, so the blocklist setter, the default admin, and the operators of the configured external sanction lists must be trusted not to block legitimate users or to omit truly sanctioned ones.

StakedLBTC Oracle owner and Notary Consortium: `StakedLBTCOracle` is `Ownable2Step`. The owner sets the consortium address, the maximum future switch interval,

and the per-interval ratio threshold. Ratio updates are authorized by `INotaryConsortium.checkProof` and additionally bounded by the configured threshold and switch time. The reported ratio drives `StakedLBTCConverter`, so users must trust the consortium signatures and the owner-configured bounds to reflect the true staking ratio. The same oracle exposes an `AggregatorV3Interface` view used as a generic Chainlink-shaped price feed; consumers that read it accept the same trust assumptions.

Migration entrypoint on `MAWARStrategyMigrated`: The `migrate` function is callable by `DEFAULT_ADMIN_ROLE` and overwrites the underlying asset, decimals, blocklist oracle, price-per-share, rate limiter bucket, high-water mark, and management fee treasury without checking that the contract has not already been migrated. The intended operational flow is to upgrade the existing `ERC4626VaultWrapper` proxy to this contract, call `migrate` to initialize the new storage, and then upgrade the same proxy to `MultiAssetWithAsyncRedemptionStrategy` in the same transaction so that the migration entrypoint is no longer reachable. Users must trust the default admin to perform these three steps atomically and to not re-invoke `migrate` if the transient implementation is ever exposed, since re-invocation would reset the rate limiter to zero capacity and the high-water mark to the current price, with downstream effects on fees and rate limiting. The migration also changes the declared `asset()` (for example, from `LBTCv` held by the legacy wrapper to `BTCB`), so until the default admin converts the strategy's existing reserves to match the new `asset()` denomination the strategy is not fully backed; redemption fulfillment cannot proceed in this window, and any `requestRedeem` calls accepted during it are priced at whatever `pricePerShare` is in effect at request time rather than the value implied by the post-conversion reserves. After `migrate`, the price rate limiter is left fully closed (`maxBps = 0`, `refillRate = 0`); `setPricePerShareRateLimit` must be called with operationally appropriate values before any `postPricePerShare` invocation can succeed, otherwise updates revert or, under `revertOnRateLimit = false`, drain the bucket and pause deposit and redeem.

Shard implementers and integrators: The shard model assumes that any externally-effective ERC-20 or native operation performed by a shard (token approvals, EIP-2612 `permit`, swaps, wrap and unwrap, and similar) is routed through `exec` so that the same `(to, value, data)` is presented to the validator. Adding parallel public surfaces on shard implementations or upgraded beacons that perform such operations without going through `exec` would bypass the allowlist policy; integrators deploying additional shard types or upgrades are expected to preserve this discipline.

StakeAndBake integration: `MultiAssetDepositStrategyDepositor` accepts calls only from a single immutable `stakeAndBake` address set at deployment. It pulls the deposit asset from that caller, grants an allowance to the strategy, and forwards a deposit with a minimum-

shares-out value supplied by the caller. Users of the StakeAndBake flow must trust that contract to forward the correct receiver and slippage parameters, while the strategy retains the same checks (blocklist, fee, converter) that apply to direct deposits.

Unprivileged participants: Depositors, redeemers, and share holders interact through the public deposit, mint, transfer, and `requestRedeem` paths. They are subject to the blocklist oracle, the strategy and transfer pauses, the configured deposit and redeem fees, the converter behavior of the chosen deposit asset, and the posted `pricePerShare` at the moment of their action.

Critical Severity

C-01 Migration Hardcodes Initial Price-Per-Share, Detaching Accrued Yield From Existing Holders

The legacy `ERC4626VaultWrapper` deployed at `0x3a4b...62F3` accounts for shares 1:1 with `LBTCv`, a Veda BoringVault share whose value in `BTCB` is published by the Veda accountant at `0x2863...3dCE`. The wrapper does not track the `LBTCv` to `BTCB` rate on-chain because redemption historically flows through Veda at the time of withdrawal.

The MAWARStrategyMigrated.migrate function redenominates the strategy: it overrides asset to `BTCB` and initializes `pricePerShare to 10 ** decimals` (i.e. one share equals one `BTCB` satoshi) without consulting the Veda accountant. Immediately after, on line 81, `highWaterMark` is stamped to the same hardcoded value via `pricePerShare`. The accountant's `getRateInQuoteSafe` call for `BTCB` currently (block 25094989) returns `102403319`, meaning one `LBTCv` is worth `1.02403319 BTCB`. With approximately 50,427,795,796 shares outstanding, the strategy will report `totalAssets` of about `504.28 BTCB` while continuing to hold `LBTCv` redeemable for approximately `516.40 BTCB`. The delta of roughly `12.12 BTCB`, about 2.4% of net asset value, is silently detached from existing holders the moment migrate is executed, with no event surfacing the mispricing.

Every existing `BTCe` holder is affected proportionally, and any holder who redeems before a corrective price update receives only the understated value. Subsequent `BTCB` depositors enter at the artificially low net asset value, permanently locking in their share of the dilution. Repricing after migration is constrained: `postPricePerShare` is governed by the rate limiter, and migrate initializes that limiter with `maxBps` and `refillRate` both set to zero, so an additional `setPricePerShareRateLimit` transaction is required before any correction can be posted. Redemptions fulfilled during this window crystallize the loss for those holders.

The hardcoded `highWaterMark` compounds the issue along a second channel. When the operator later raises `pricePerShare` toward the true accountant rate, FeeManagerUpgradeable._harvestPerformanceFee observes `pps > hwm` and treats the correction as new performance, minting fee shares to the treasury proportional to `(pps - hwm) / hwm * performanceFeeBps`. The yield being "harvested" already accrued under the legacy wrapper and was already credited to BTCe holders; the fee is purely an artifact of the artificially low high-water mark. If a non-zero `performanceFeeBps` is configured before

the correction is posted, the treasury captures a second slice of the same ~2.4% NAV gap that existing holders already absorbed at the moment of migrate.

Consider passing the initial price-per-share to migrate as an explicit parameter sourced from the Veda accountant in the same transaction or block as the upgrade, bounding the accepted value to a sanity range so a misbehaving accountant cannot set an arbitrary `pricePerShare`, and initializing `highWaterMark` from that same accountant-derived value rather than from the post-set `pricePerShare`. If the design intent is for the treasury to capture the accrued `LBTCv` yield rather than crediting it to `BTCe` holders, this should be an explicit, documented decision rather than an implicit consequence of the hardcoded initializer.

Update: *Resolved in pull request #453 at commit 9d60ce0.*

Medium Severity

M-01 Manager Role Can Bypass Admin-Only Converter Updates by Re-Listing a Deposit Asset

`MultiAssetDepositUpgradeable` supports multiple deposit tokens by converting `depositAsset` amounts to strategy base units through `IAssetConverter`, then converting base units to shares via `pricePerShare`. Share issuance in `deposit` depends directly on the configured converter output through `previewDeposit`, so the converter address governs deposit pricing.

Converter updates are intended to be admin-governed through `setAssetConverter`, which is restricted to `DEFAULT_ADMIN_ROLE`. However, `removeDepositAsset` and `addDepositAsset` are gated by `MANAGER_ROLE`. Removal deletes `$.converters[depositAsset]`, and re-adding re-registers the asset under a converter chosen by the manager. The sequence executes entirely under `MANAGER_ROLE`: 1. A token `T` is listed with converter `C1`. 2. The manager calls `removeDepositAsset(T)`, which clears the stored converter for `T`. 3. The manager calls `addDepositAsset(T, C2, feeBps)`, which re-lists `T` under the manager-chosen converter `C2`.

Deposits of `T` are then priced using `C2`, bypassing the role separation that `setAssetConverter` is intended to enforce. Because share issuance depends on converter output, this expands the blast radius of a compromised or operationally hot manager to deposit pricing, which was scoped to `DEFAULT_ADMIN_ROLE`.

Consider aligning access control so the same trust domain governs `allowlisting` and pricing, for example by restricting `addDepositAsset` and `removeDepositAsset` to `DEFAULT_ADMIN_ROLE`, or by enforcing that re-adding a previously listed asset cannot change `$.converters[depositAsset]` without an explicit admin action such as an admin-confirmed converter change or a time-delayed update.

Update: *Resolved. Instead of updating `addDepositAsset` and `removeDepositAsset` to be restricted to `DEFAULT_ADMIN_ROLE`, `setAssetConverter` was original meant to be restricted to `MANAGER_ROLE`. This is reflected in pull request #454 at commit bc88f32.*

M-02 Fee Share Minting Inflates Reported NAV Because `pricePerShare` Is Not Adjusted

The strategy stores `pricePerShare` as a posted exchange rate that is updated only through the rate-limited `postPricePerShare` entry point, and the protocol reports `totalAssets` as `totalSupply × pricePerShare`. Fee accrual relies on this same posted rate: `_harvestPerformanceFee` compares `pricePerShare()` to the stored high-water mark, computes the fee size off the pre-mint share supply, and mints shares to the treasury, while `_accrueManagementFee` similarly mints management fee shares against the current supply.

Neither function reduces `pricePerShare` after the mint. As a result, the share supply grows but the per-share rate stays the same, so the strategy's posted NAV strictly increases even though no underlying assets are added. Concretely, with `totalSupply = 100`, `pricePerShare = 1.10` (asset wei per share unit), and `highWaterMark = 1.00`, a 20% performance fee mints two shares to the treasury. After the mint, `totalSupply × pricePerShare = 102 × 1.10 = 112.2`, but the strategy still custodies only `110` units of underlying value. The 2.2 unit gap is fabricated NAV that downstream operations now rely on.

This corrupts every flow that consumes `pricePerShare`. Subsequent depositors mint shares at an overstated rate and receive fewer shares than the underlying they contribute warrants. Subsequent redeemers exit at the same overstated rate, draining underlying faster than the share supply implies and pushing the loss onto whoever redeems last. The high-water mark is also set to the pre-mint `pricePerShare`, so the next harvest sees the same stale rate as a valid reference point, compounding the discrepancy with each accrual. The same effect applies to management fee mints, which occur on every deposit and redemption flow through `MultiAssetDepositUpgradeable` and `AsyncRedemptionUpgradeable`.

The same root cause surfaces in the separate v2 implementation of `postPricePerShare`, which introduces an explicit reconciliation step against an operator-supplied NAV snapshot. When `currentSupply` exceeds `supplyAtSnapshot`, that version treats `sharesAdded = currentSupply - supplyAtSnapshot` as deposit-driven supply growth and credits the gap as `sharesAdded × pricePerShare / oneShare` toward `reconciledNav`. The reconciliation is sound for deposits and async redeem requests, since both move shares against matching asset flows priced at the stale PPS, but fee shares minted between the off-chain snapshot and the on-chain post carry no asset inflow and are indistinguishable from deposits to this arithmetic. They therefore compound into `reconciledNav` and propagate the same inflated PPS forward. For instance, with `pricePerShare = 1` and `totalSupply = 100`, an operator snapshot of `navAtSnapshot = 100, supplyAtSnapshot = 100` followed by a 10-share management fee mint and a `postPricePerShare(100, 100, ...)` call yields `sharesAdded = 10` and `reconciledNav = 100 + 10 × 1 = 110`, leaving the stored PPS at 1 even though real backing NAV is still 100. The error is also self-reinforcing across both implementations: performance fee mints based on the inflated PPS update the high-water mark to that PPS, so subsequent posts can compound the discrepancy.

Consider, immediately after minting fee shares in `_harvestPerformanceFee` and `_accrueManagementFee`, updating the stored `pricePerShare` so that `newTotalSupply × newPricePerShare` equals the pre-mint NAV (`oldTotalSupply × oldPricePerShare`). As the update is a mechanical consequence of an internal mint and does not reflect a change in underlying value, consider routing it through an internal setter that bypasses the `PriceRateLimiter` used by `postPricePerShare`, and emitting an event that distinguishes fee-induced rebases from oracle-posted price updates. This adjustment neutralizes the failure mode in both v1 and v2: once dilution is reflected in the stored PPS at mint time, the v2 reconciliation can no longer misattribute fee shares as deposits.

Update: *Acknowledged, not resolved. Relating to the v1 implementation the Lombard team stated:*

> Our solution is: - Correcting PPS on the next external post preserves a single, auditable path for exchange-rate changes. - Fee parameters are governed so total fees (management + performance + flow fees) remain **below yield/rewards** accruing to remaining holders; otherwise dilution would exceed economic gain.

The Lombard team added further documentation to clarify posted NAV behavior in pull request #475 at commit e102921.

For the v2 implementation, the Lombard team stated:

> Posted NAV may therefore overstate economic backing between fee events and the next operator post. That gap is closed on the **next** `postPricePerShare`, when the operator supplies current `supplyAtSnapshot` and true backing `navAtSnapshot`, and reconciliation sets PPS for the full outstanding supply (including fee shares).

The Lombard team added further documentation to clarify fee minting impact on totalAssets and NAV in pull request #474 at commit 79ed4ac.

M-03 Stale `pricePerShare` Lacks On-Chain Staleness Bound

The strategy's net asset value accounting relies on a single `pricePerShare` value stored in `StrategyBaseUpgradeable`, which is posted off chain by an account holding the `PRICE_SETTER_ROLE` through `postPricePerShare`. The rate limiter caps how much the value can move per update, but it does not constrain how often the value must be refreshed.

Both `_convertToShares` and `_convertToAssets` read this stored value with no notion of when it was last set. If the off chain poster halts (for example, due to an operational outage, key loss, paused operations, or any other condition that prevents posting), conversions continue indefinitely at the last posted rate. As a consequence, share mint and redemption accounting can drift from the true underlying NAV while deposits and redemptions remain open at a price that no longer reflects strategy performance. Depositors entering during a stale window receive shares priced against an outdated NAV, and redeemers settling against a stale NAV are paid an amount that may diverge from the real value of their shares, with the direction of the drift determining which side absorbs the loss.

Although `PricePerShareUpdated` emits `block.timestamp` and the `PRICE_SETTER_ROLE` is trusted to keep the price fresh, on-chain reads have no defense if that role fails to act.

Consider sending the update timestamp in the `postPricePerShare` call and storing the value, and exposing a configurable maximum staleness window. When the window is exceeded, consider reverting deposit and redemption flows that depend on `_convertToShares` or `_convertToAssets`, or automatically pausing the affected entry points, so that conversions cannot silently proceed against an outdated rate.

Update: *Acknowledged, not resolved. The Lombard team stated:*

> Intended, answer could be: Stale NAV is real if `PRICE_SETTER` stops posting — conversions use the last `pricePerShare` until ops intervenes. That's by design: off-

> chain NAV + on-chain rate limits, with timestamps in `PricePerShareUpdated` for monitoring. We're **not** making deposits async or blocking every flow on a staleness timer; deposits stay instant, and we rely on monitoring + `STRATEGY_PAUSER` (and auto-pause on rate-limit breach) to stop new mints/redeems if posting fails.

M-04 Missing Staleness Check When Using Staked LBTC Oracle Ratio

The `StakedLBTCConverter` adapts `StakedLBTCOracle` to the `IAssetConverter` interface used by the strategy for translating between `StakedLBTC` token amounts and BTC accounting units. Both `toBaseUnits` and `fromBaseUnits` read the staking ratio by calling `oracle.ratio()`, which returns only the current ratio value without any indication of when it was last updated.

The `StakedLBTCOracle` is updated off-chain via `publishNewRatio`, and an interruption in the update cadence (for example, consortium downtime, signature service outage, or a governance pause) can leave the ratio stuck at a stale value indefinitely. Because the converter has no visibility into when the ratio was last refreshed, the strategy will continue to perform conversions against outdated yield information, mispricing deposits and withdrawals relative to the true accrued yield of the underlying staked position. The oracle already exposes the update timestamp through its `latestRoundData` function, which returns the `updatedAt` field alongside the rate, but the converter currently ignores this signal.

Consider replacing the `oracle.ratio()` calls in `toBaseUnits` and `fromBaseUnits` with a call to `oracle.latestRoundData()`, and introducing a configurable staleness threshold (settable by a privileged role) that the converter checks against `block.timestamp - updatedAt`, reverting if the ratio is older than the configured bound so that downstream strategy operations revert rather than executing against stale yield data.

Update: *Resolved, the Lombard team stated:*

> If we start using `latestRoundData` and heartbeat check. Do you think we can simply remove `StakedLBTCConverter` and use `ChainlinkConverter` for `StakedLBTCOracle`? Otherwise it seems the logic will very similar in both contracts

After further review, we agreed with the Lombard team. `StakedLBTCOracle` *is now removed in pull request #477 at commit 26dc714.*

M-05 Redemption Requests Lock Payout at Request-Time `pricePerShare`, Enabling Front-Running of PPS Updates

The strategy uses an asynchronous redemption flow: `requestRedeem` burns the owner's shares immediately and records a request, while `fulfillRedeemRequests` later transfers the underlying asset to the owner. The strategy's `pricePerShare` is updated by accounts holding `PRICE_SETTER_ROLE` through `postPricePerShare`, subject to a per-update rate-limiter cap.

In `_requestRedeem`, the asset amount owed to the user is computed at the current `pricePerShare` via `_convertToAssets(netShares, Math.Rounding.Floor)` and stored in `RedeemRequest.pendingAssets`. At fulfillment time, the contract transfers exactly that stored amount, irrespective of the `pricePerShare` prevailing when the request is settled. Because `postPricePerShare` transactions are visible in the public mempool before inclusion, an observer of a pending downward `pricePerShare` update can submit a `requestRedeem` transaction immediately beforehand to lock in the higher historical price. The expected gain per share is approximately `oldPricePerShare - newPricePerShare`, accruing over all shares the observer redeems before the update lands.

The same dynamic applies symmetrically to deposits. The `deposit` entry point mints shares synchronously at the current `pricePerShare` via `previewDeposit`, so a pending upward `pricePerShare` update can be front-run by a deposit that obtains shares at the lower pre-update price and is worth more immediately after the update is included. In both directions, the rate-limiter cap on a single `postPricePerShare` move bounds the extractable amount per update, but does not prevent the extraction itself.

Consider valuing redemption requests at the `pricePerShare` prevailing at fulfillment time rather than at request time, so that the payout reflects the protocol's state at settlement. If forward-priced redemptions are required by the design, consider mitigations that reduce mempool observability of `pricePerShare` updates, such as routing `postPricePerShare` transactions through a private mempool, introducing a commit-reveal delay between redemption requests and their pricing, or applying a small bid-ask spread to deposits and redemptions to make front-running unprofitable below a threshold move.

Addendum: existing fee surface *The codebase already exposes admin-configurable bid/ask-style fees of the kind suggested above: per-asset `depositFeeBps` on synchronous deposits, and a global `redeemFeeBps` charged in shares at request time. For a round-trip front-run (deposit at stale PPS, then redeem at the new PPS) to be unprofitable, the* ***combined*** *spread must exceed the per-post rate-limiter cap on `|ΔPPS / PPS|`:*

```
depositFeeBps + redeemFeeBps  ≥  ratelimiter_cap_bps
```

Update: *Acknowledged, not resolved. The Lombard team stated:*

> We accept this attack and want to mitigate it by fees and daily ratio updates.

Low Severity

L-01 Migration Accepts Decimals Parameter Without Cross-Checking the Underlying Asset

The `MAWARStrategyMigrated.migrate` function accepts the underlying asset's decimal precision as a `uint8` parameter and stores it directly in `baseStorage.underlyingDecimals`, validating only that the value is non-zero. The inherited `_tryGetAssetDecimals` helper on `StrategyBaseUpgradeable` already retrieves this value from the asset on-chain and is used by the standard `__StrategyBase_init` path, but it is not consulted here. If an incorrect value is supplied, every subsequent share-to-asset and asset-to-share conversion is scaled by a factor of `10 ** delta` against the asset's true decimals, mispricing all deposits and redemptions until corrected.

Consider invoking `_tryGetAssetDecimals` on the supplied `asset_` inside `migrate` and reverting when the returned value either fails to decode or does not match the supplied `decimals_`, mirroring the validation already applied by `__StrategyBase_init`.

Update: *Acknowledged, not resolved. The Lombard team accepts this as an operational risk.*

L-02 Migration Function Lacks Idempotency Guard, Allowing Silent Accounting State Reset on Repeated Invocation

The `MAWARStrategyMigrated.migrate` function is documented as a one-time entry point, called immediately after the upgrade from `ERC4626VaultWrapper` to initialize the new strategy storage. The function is gated by `onlyRole(DEFAULT_ADMIN_ROLE)` but contains no on-chain check enforcing single use. A second invocation by any account holding the admin role would execute the full body again.

A repeated call overwrites accounting state that, by the time of the second call, would generally have diverged from its initial values. The `pricePerShare` field is reset to `10 ** decimals()`, the `FeeManager` high-water mark is reset to the resulting `pricePerShare`, and the price rate limiter configuration is reset to `maxBps` and `refillRate` both equal to zero. The `asset`, `underlyingDecimals`, `blocklistOracle`, and `managementFeeTreasury` fields are rewritten from the second call's calldata. None of these updates are distinguished by a dedicated event, and the events that are emitted are identical to those of the first legitimate migration, so off-chain monitoring would not naturally surface the second execution as anomalous.

The practical consequences depend on the interval between the two calls. If any `pricePerShare` update has occurred in the meantime, the second call would reset the share price to par and reprice every outstanding share against the lower value, while also resetting the high-water mark and forfeiting any unharvested performance fees accrued under the prior mark. The price rate limiter would return to a fully closed state, undoing operational calibration performed after the first migration. The path to a second execution does not require active misuse: a timelock queue containing two copies of the same proposal, a multisig coordination error producing duplicate confirmations, or an operational re-submission after a transaction is incorrectly believed to have failed would each result in a second on-chain execution.

Consider adding a one-shot guard inside `migrate` that reverts on any invocation after the first, for example by reverting when `baseStorage.asset` is already non-zero, or by introducing a dedicated `migrated` flag in a namespaced storage slot. This would align the function with the one-shot initialization pattern established by the `Initializable` contract and remove the dependency on operator hygiene for correctness.

Update: *Acknowledged, not resolved. The Lombard team stated:*

> We will do 1 call and then upgrade to the usual implementation to prevent the ability to call it twice

L-03 Zero-Asset Redemption Requests Permanently Trap Burned Shares

In `AsyncRedemptionUpgradeable._requestRedeem`, the asset amount owed to the requester is computed by `_convertToAssets(netShares, Math.Rounding.Floor)` and stored in `rq.pendingAssets`. The requester's net shares are burned in the same call, while the recorded `pendingAssets` value is the only ledger entry that drives later payout.

When the price per share is sufficiently low relative to `10 ** decimals`, the floor rounding of `_convertToAssets` can return `0`. The request is still created and the net shares are still burned, but `pendingAssets` is recorded as `0`. Any subsequent call to `fulfillRedeemRequests` reverts with `AsyncRedemption_NothingToFulfill` for that request identifier, and there is no cancellation or refund mechanism that would allow the requester to recover the burned shares. The burned shares are permanently lost while the request entry persists in storage and can never be discharged.

Consider reverting in `_requestRedeem` when the computed `assets` value is zero, for example by raising a dedicated `Strategy_ZeroAmount` error after the conversion, so that the request is rejected before any shares are burned and no unfulfillable entry is created.

Update: *Resolved in pull request #455 at commit e0a20c4.*

L-04 `postPricePerShare` Cannot Update Price When Deposits or Redemptions Are Paused

The `postPricePerShare` function in `StrategyBaseUpgradeable` is gated by the `whenDepositNotPaused`, `whenRedeemNotPaused`, and `whenNotPaused` modifiers. As a result, if either deposits or redemptions are independently paused, for example to apply a temporary operational hold on a single side of the strategy while a converter or external dependency is being addressed, the `PRICE_SETTER_ROLE` is unable to post new exchange rates. The posted price per share then becomes stale relative to the underlying NAV. A stale price per share affects every path that uses `_convertToShares` or `_convertToAssets`.

Consider relaxing the modifier set on `postPricePerShare` so that it can post updated prices while deposits or redemptions are individually paused.

Update: *Resolved in pull request #456 at commit f93df5d.*

L-05 Dead Code in `StrategyBaseUpgradeable`

The `StrategyBaseUpgradeable` contract defines an internal `_pauseDepositAndRedeemForRateLimit` helper whose NatSpec describes it as the path for the PPS soft overflow case and which sets both pause flags without the `whenDepositNotPaused` and `whenRedeemNotPaused` modifiers. This function is however never used anywhere in the codebase.

Consider removing the `_pauseDepositAndRedeemForRateLimit` function to increase code clarity code readability.

Update: *Resolved in pull request #457 at commit d8ddbf9.*

L-06 `convertToAssets` Uses Non-Standard Rounding Direction

In the `MultiAssetDepositUpgradeable` contract, the `convertToAssets` view function converts shares to base units with `Math.Rounding.Floor` and then converts the resulting base amount to deposit-asset units with `Math.Rounding.Ceil`. The ERC-4626 specification requires `convertToAssets` to round down so that the returned value does not overstate the assets redeemable for a given share amount. As a result, the function may return a value slightly larger than what is actually obtainable, which can mislead integrations that rely on standard ERC-4626 rounding semantics. The function is `view`-only and does not affect on-chain accounting.

Consider applying `Math.Rounding.Floor` to the `fromBaseUnits` conversion so that `convertToAssets` aligns with the ERC-4626 rounding convention.

Update: *Resolved in pull request #473 at commit 21a1135.*

L-07 Management Fee Accrues While Protocol Is Paused

In `FeeManagerUpgradeable`, `_accrueManagementFee` is only invoked from the deposit and redemption entry points, both of which are gated by `whenDepositNotPaused` and `whenRedeemNotPaused`. While both flows are paused, `lastFeeHarvestAt` is not advanced, so the first deposit or redemption after resume mints fee shares to `managementFeeTreasury` for the entire elapsed window in a single transaction. As an illustration, a 30-day pause at a 2% annual rate translates into roughly 0.16% lump-sum dilution.

Consider pausing accrual of management fees when the protocol is paused to prevent unfair dilution of users.

Update: *Acknowledged, not resolved. The Lombard team stated:*

> Intended behaviour. After the vault is paused, we can decide operationally to set 0 fees

With `setManagementFeeBps` now accruing at the prior rate before writing the new bps (per the separate fix), the operational pattern (pause -> `setBps(0)` -> unpause -> restore) should work correctly.

L-08 Management Fee Compounds On Treasury Share Balance

`FeeManagerUpgradeable` accrues the periodic management fee by minting new shares to a configured treasury at a rate set in basis points per year. Inside `_accrueManagementFee`, the share count is computed by `_computeManagementFeeShares` using the current `totalSupply()` as the base, and the resulting shares are minted to `managementFeeTreasury`.

Because the freshly minted shares are added to `totalSupply` and the treasury continues to hold them, each subsequent invocation of `_accrueManagementFee` charges the configured rate on a base that already includes every prior accrual minted to the treasury. The management fee therefore compounds on its own previously minted shares rather than accruing linearly against the underlying user supply.

Consider excluding the treasury's own share balance from the base used in `_computeManagementFeeShares`, so that fee shares previously minted to the treasury do not feed into subsequent accruals.

Update: *Acknowledged, not resolved. The Lombard team stated:*

> This matches the intended dilution model: the nominal `managementFeeBps` is an annual rate applied to outstanding shares, and fee shares are first-class supply. Excluding the treasury balance would under-accrue relative to that model.

Lombard team also updated docstrings to clarify management fee accrual logic in pull request #472 at commit 8a0678d.

L-09 `maxDeposit` and `maxMint` Ignore ERC-20 Transfer Pause

`StrategyBaseUpgradeable` maintains three independent pauses: the deposit and mint pause exposed via `depositPaused`, the redemption pause exposed via `redeemPaused`, and the ERC-20 transfer pause inherited from `ERC20PausableUpgradeable` and exposed

via `paused`. When the ERC-20 transfer pause is active, share transfers revert, including the `_mint` call invoked during a deposit.

In `MultiAssetDepositUpgradeable`, `maxDeposit` and `maxMint` only return `0` when the asset is unlisted or `depositPaused` is set. They do not consult the ERC-20 transfer pause. As a result, while the share token is paused, both functions report `type(uint256).max` even though every call to `deposit` or `mint` reverts inside `_mint` due to the pause. This contradicts the ERC-4626 requirement that `maxDeposit` and `maxMint` must factor in any condition that disables deposits, including temporary ones, and may mislead integrators, aggregators, and front ends that rely on these helpers to size deposits.

Consider returning `0` from `maxDeposit` and `maxMint` whenever the ERC-20 transfer pause is active, in addition to the existing `depositPaused` and listing checks.

Update: *Resolved in pull request #458 at commit 6ff85c6.*

L-10 Merkle Allowlist Constraints With Zero `expected` Match Against Truncated Calldata

`MerkleAllowlistValidator` authorizes calls by checking, for each constraint in a rule, that a 32-byte window of the call's argument region matches `expected` under a `mask`. The window is read by `_wordAt`, which is documented to right-pad with zero when the requested offset lies past the end of `data`. The matching loop in `_constraintsMatchCalldata` then compares `_wordAt(data, cOff + 4) & mask` against `expected`.

When a constraint has `expected == bytes32(0)` and the rule does not set the `FLAG_DATA_LENGTH` flag, the comparison succeeds even if the calldata is too short to reach the offset. For any such offset, `_wordAt` returns `bytes32(0)`, the masked value is `bytes32(0)`, and the constraint matches. The rule was likely written to mean "the masked region at this position must be zero", but it silently degrades to "the masked region must be zero or absent". When `FLAG_DATA_LENGTH` is set, the length check preempts this path, but the flag is optional and rule authors are not forced to use it.

In most cases, the target contract reverts before any state change because truncated calldata fails ABI decoding or falls through to a reverting fallback. Targets with non-standard calldata handling, such as multicall wrappers, low-level routers, or assembly-heavy entrypoints, may instead interpret the truncated payload as a different but valid operation, allowing a call that the rule author did not intend to authorize.

Consider requiring constraints to be reachable by the calldata at match time. This can be enforced inside `_constraintsMatchCalldata` by returning `false` whenever the offset is outside the calldata array.

Update: *Resolved in pull request #459 at commit 82ccf4e.*

L-11 Merkle Calldata Constraints Do Not Model ABI Decoding for Dynamic Arguments

`MerkleAllowlistValidator` enforces rule constraints as fixed 32-byte windows at `4 + calldataOffset` in `_constraintsMatchCalldata`, and does not validate ABI relationships between head words and dynamic tails. Because Solidity ABI decoding accepts non-canonical but in-bounds dynamic offsets and lengths, a rule that pins bytes inside a dynamic argument without also pinning the corresponding ABI head words can be satisfied by `calldata` in which the callee decodes the dynamic argument from a different region than the rule author intended.

The specification and operational guidance describe how to compute constraint offsets for dynamic types (see AGENTS.md and 001-merkle-call-data-allowlist.md), but they frame this only as a computation note and do not state the soundness requirement that the corresponding head words must also be pinned. As a result, a rule author following the documented guidance can produce a rule whose pinned bytes do not bind the value the callee actually decodes.

Consider documenting directly in the validator contract that rules constraining bytes inside a dynamic argument must additionally pin the corresponding ABI head words (offset and length) and, where feasible, the exact `data.length`, and mirroring this requirement in the specification and authoring guidance so that rule authors have the soundness condition available at the point of use.

Update: *Resolved in pull request #471 at commit 0397020.*

L-12 `setManagementFeeBps` Retroactively Applies New Rate To Unharvested Interval

`_accrueManagementFee` computes fee shares as bps * (block.timestamp - lastFeeHarvestAt) using the current `managementFeeBps`. Because `setManagementFeeBps` updates the rate without first calling `_accrueManagementFee`,

the new rate is applied to the entire interval since the previous accrual, not just to time elapsed after the configuration change.

The interval since the last accrual is bounded only by deposit and redeem activity and can extend for arbitrary periods of low activity. An administrator raising the rate therefore retroactively reprices fees over time when users were operating under the prior rate, which contradicts the forward-looking reading suggested by the existing NatSpec that "first accrual at this rate happens on the next `_accrueManagementFee` call." The performance fee setter already settles accrued fees at the prior configuration before updating the rate.

Consider invoking `_accrueManagementFee` at the start of `setManagementFeeBps` so the prior interval is harvested at the prior rate, and the new rate only applies to time elapsed after the change.

Update: *Resolved at commit 3a34f90.*

Notes & Additional Information

N-01 Blocklist Enforcement Gaps Allow Sanctioned Value to Exit Through Redemption

The blocklist oracle is enforced at the deposit entry point and at submission of `requestRedeem`, where `_requireBlocklistAllows(owner)` is checked. Two gaps in this enforcement allow value associated with a blocklisted party to be paid out through the redemption flow.

First, share-token balances are not frozen, and the share contract permits standard ERC-20 transfers regardless of the sender's blocklist status. A blocklisted holder can transfer their shares to an address that is not blocklisted, and that recipient can then call `requestRedeem` on its own behalf. The submission check on `owner` passes because the new owner is unrestricted, and the underlying asset is subsequently paid out to the recipient by the `PAY_REDEMPTIONS_ROLE`.

Second, `fulfillRedeemRequests` does not re-check the blocklist at payout. If `owner` becomes sanctioned between submission and fulfillment, whether through a sanction-list

update or a manual block, no on-chain enforcement prevents the payout, and only off-chain operator coordination would withhold settlement.

Consider enforcing the blocklist within the share-token transfer hook so that transfers involving a blocklisted sender or recipient are rejected, and re-checking the blocklist for `owner` and receiver inside `fulfillRedeemRequests` so that any blocklist status change between submission and payout is honored on-chain. If the functionality is intended, consider documenting it as such.

Update: *Resolved in pull request #506 at commit 0b3d668 for the share-token transfers instance. Relating to the `fulfillRedeemRequests` instance, the Lombard team stated:*

> We deliberately perform the blocklist check at request time rather than at fulfillment to save gas, and to allow an offline review window. If an address is blocklisted in between, that case is handled through a separate KYC/recovery flow where the user can work with us to retrieve their tokens.

N-02 `ShardBaseUpgradeable` NatSpec Misrepresents Validator as Immutable

The NatSpec for `ShardBaseUpgradeable` states that values are set once in the initializer and never changed. However, `setValidator` allows an account with `DEFAULT_ADMIN_ROLE` to replace the validator at any time. Because exec reads the validator on every call, a replacement can take effect between off-chain proof generation and on-chain execution.

Consider correcting the NatSpec to reflect that the validator is mutable.

Update: *Resolved in pull request #469 at commit 407148f.*

N-03 Missing Security Contact Information

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is beneficial because it allows the code owners to define the communication channel for vulnerability disclosure, reducing the risk of miscommunication or failure to report due to uncertainty about how to proceed. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their

maintainers to contact the appropriate person about the problem and provide mitigation instructions.

All contracts under the `strategy` subdirectory were identified as not having a security contact.

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the `@custom:security-contact` convention is recommended as it has been adopted by the OpenZeppelin Wizard and ethereum-lists.

Update: *Resolved in pull request #468 at commit 0543d4c.*

N-04 Unused Errors

Within `MultiAssetDepositStrategyDepositor.sol`, the following errors are defined but never used:

- The `ZeroAssets` error
- The `ApproveFailed` error
- The `MinimumMintNotMet` error

To improve the overall clarity, intentionality, and readability of the codebase, consider either using or removing any currently unused errors.

Update: *Resolved in pull request #467 at commit 7cfd740.*

N-05 Unused Imports

Within `MultiAssetWithAsyncRedemptionStrategy.sol`, there are imports that are unused and could be removed.

- The import `import {FeeManagerUpgradeable} from "./FeeManagerUpgradeable.sol";` .
- The import `import {StrategyBaseUpgradeable} from "./StrategyBaseUpgradeable.sol";` .

Consider removing unused imports to improve the overall clarity and readability of the codebase.

Update: *Resolved in pull request #466 at commit 0333827.*

N-06 Constant Not Using `UPPER_CASE` Format

In `MAWARStrategyMigrated.sol`, the `ReentrancyGuardStorageLocation` constant is not declared using `UPPER_CASE` format.

According to the Solidity Style Guide, constants should be named with all capital letters with underscores separating words.

Consider updating the name of `ReentrancyGuardStorageLocation` to follow this convention for better readability and consistency.

Update: *Resolved in pull request #465 at commit b32d567.*

N-07 Variables Initialized With Their Default Values

Throughout the codebase, multiple instances of variables being initialized with their default values were identified:

- In `BlocklistOracle.sol`, the `i` variable
- In `BlocklistOracle.sol`, the `i` variable
- In `MerkleAllowlistValidator.sol`, the `i` variable

To avoid wasting gas, consider not initializing variables with their default values.

Update: *Resolved in pull request #464 at commit faf62a2.*

N-08 Inconsistent CapWords Style

Throughout the codebase, there are multiple components that do not follow the Solidity style guide.

- Several errors of the `ChainlinkConverter` contract
- Several errors of the `StakedLBTCConverter` contract
- The `PriceRateLimiter_RateLimitExceeded` error

Consider adhering to the Solidity style guide by naming contracts, structs, enums, events, and errors using the CapWords style to improve the project's overall legibility.

Update: *Resolved in pull request #463 at commit 94d41e4.*

N-09 Unused Named Return Variables

Named return variables are a way to declare variables that are meant to be used within a function's body for the purpose of being returned as that function's output. They are an alternative to explicit in-line `return` statements.

Throughout the codebase, multiple instances of unused named return variables were identified:

- In `AsyncRedemptionUpgradeable.sol`, the `totalPendingShares` return variable of the `totalPending` function
- In `AsyncRedemptionUpgradeable.sol`, the `totalPendingAssets` return variable of the `totalPending` function
- In `AsyncRedemptionUpgradeable.sol`, the `assets` return variable of the `pendingAssetsOf` function
- In `AsyncRedemptionUpgradeable.sol`, the `requestId` return variable of the `requestRedeem` function
- In `BlocklistOracle.sol`, the `lists` return variable of the `getSanctionLists` function
- In `StakedLBTCConverter.sol`, the `baseOut` return variable of the `toBaseUnits` function
- In `StakedLBTCConverter.sol`, the `tokenOut` return variable of the `fromBaseUnits` function
- In `MerkleAllowlistValidator.sol`, the `word` return variable of the `_wordAt` function

Consider either using or removing any unused named return variables.

Update: *Resolved in pull request #462 at commit 14dd592.*

N-10 Missing `@custom:storage-location` Annotation on `StakedLBTCOracleStorage` Struct

The `StakedLBTCOracleStorage` struct uses an ERC-7201 namespaced storage location computed from `lombardfinance.storage.StakedLBTCOracle` but lacks the corresponding `/// @custom:storage-location erc7201:lombardfinance.storage.StakedLBTCOracle` NatSpec annotation, which upgrade-safety tooling such as `@openzeppelin/upgrades-core` relies on to track the struct's storage layout and detect collisions during upgrades.

Consider adding the `/// @custom:storage-location erc7201:lombardfinance.storage.StakedLBTCOracle` annotation above the struct so that automated upgrade-safety checks include it in storage layout analysis.

Update: *Resolved in pull request #461 at commit 12b3890*

N-11 Merkle Allowlist Proofs Remain Valid Across Upgrades At Allowlisted Targets

The `MerkleAllowlistValidator` authorizes shard calls by verifying a Merkle proof of a `ruleEncoded` payload under `allowlistRoot`, then matching the call against decoded fields such as `to`, `selector`, `value`, and masked-byte constraints. The rule format binds the target only by `address to`, and `_leaf` commits to the domain separator and rule bytes without incorporating any code identity.

Rule matching checks the target by address equality, so a proof issued against an upgradeable target remains valid across implementation changes that preserve the address. If an allowlisted `to` is a proxy (or otherwise mutable through beacon updates, CREATE2 redeploys, or metamorphic patterns) and is upgraded to an implementation that retains the selector but alters its effects, the previously issued proof still verifies. Since `ShardBaseUpgradeable.exec` forwards the original `data` and `value` once `validator.isAllowed` returns true, the post-upgrade behavior becomes reachable from shard custody without any change to `allowlistRoot`.

Consider treating address-and-calldata allowlisting as a trust boundary: restrict allowlisted targets to immutable contracts or explicitly trusted upgrade domains, and rotate `allowlistRoot` whenever an allowlisted target's semantics can change. Alternatively, consider introducing a rule version that pins proxy-aware code identity (for example, the

EIP-1967 implementation or beacon address, optionally with the implementation `extcodehash`) so proofs cannot outlive relevant upgrades absent an intentional root update.

Update: *Acknowledged, not resolved. The Lombard team stated:*

> This is operational risk we accept.

Client Reported

CR-01 NAV Inflation From Supply Changes Between Snapshot And Price Update Allows Over-Claims Against Real Backing

The `StrategyBaseUpgradeable` contract exposes `postPricePerShare`, which accepts a precomputed `newPricePerShare` value derived by the off-chain operator from a NAV snapshot taken at some earlier time. The function stores that price directly and applies it uniformly to the current `totalSupply` when computing `totalAssets`. The precomputed price, however, reflects the supply observed at the snapshot, not the supply present at execution.

Between the moment the operator captures the snapshot and the moment the transaction is included on-chain, the contract continues to accept synchronous deposits and asynchronous redemption requests. These mints and burns are settled against the still-stored prior `pricePerShare`, which is the price the depositing or redeeming user actually transacted at. When the operator transaction is then mined and the new `pricePerShare` is applied to the now-changed supply, the resulting `totalAssets` no longer matches the real asset backing held by the strategy.

For upward revisions, the discrepancy is positive: `totalAssets` is inflated relative to real backing. A deposit at the stale prior price mints shares against backing valued at that prior price, but once the new price is posted those shares are revalued at the higher price along with the rest of the supply. The on-chain bookkeeping then records aggregate claims that exceed the assets actually held by the strategy. Synchronous redemptions, and first-served asynchronous redemption fulfillments, draw against the inflated `totalAssets` at the inflated rate and succeed in full, while later redeemers encounter a shortfall.

Because `postPricePerShare` transactions are visible in the public mempool and the off-chain NAV report is typically observable before submission, no privileged access is required to

trigger the discrepancy. A searcher that detects an upcoming upward revision can deposit at the stale price first, and the supply mismatch then produces the over-claim automatically. A user who happens to deposit in the same window has the same effect without intent. The `PriceRateLimiter` bounds the magnitude of any single price move, but does not prevent the inflation, because the inflation arises from the supply change being unaccounted for rather than from the size of the price move itself.

Update: *Resolved in PR 442 at commit `da44075`. `postPricePerShare` now accepts (`navAtSnapshot, supplyAtSnapshot, revertOnRateLimit`) and reconciles the snapshot NAV against the current supply by valuing intervening mints and burns at the still-stored prior `pricePerShare` before dividing by the current supply. The resulting stored `pricePerShare` multiplied by `totalSupply` matches the strategy's real asset backing, eliminating the over-claim. The rate limiter continues to gate on the declared move, and rounding directions bias the reconciled price downward in both the supply-increase and supply-decrease branches.*

Conclusion

The Lombard Strategies codebase implements a yield-bearing strategy share token whose unit value tracks an off-chain NAV posted on chain, with multi-asset deposits, asynchronous redemptions, an operational shard model with validator-gated execution, a compliance oracle, and an adapter that bridges the existing StakeAndBake flow into the new strategy. OpenZeppelin audited the strategy core and its deposit, fee, and redemption modules, the shard contracts and their Merkle allowlist validator, the asset converters, the blocklist and stakedLBTC oracles, the StakeAndBake depositor, and the one-shot migration variant intended to upgrade the legacy `ERC4626VaultWrapper` deployment.

The codebase is organized as a modular composition of upgradeable contracts with ERC-7201 namespaced storage and explicit separation between configuration, pricing, and execution roles, and the shard execution model in particular layers a Merkle-rooted allowlist, calldata constraints, and a domain separator bound to the deployment over a privileged-executor bypass. The audit nonetheless surfaced concerns concentrated in three areas: the migration entrypoint, NAV and price-per-share accounting under the fee-share and asynchronous-redemption flows, and the absence of on-chain staleness bounds on both the posted price and the stakedLBTC oracle ratio. Several issues in the Merkle calldata-constraint mechanism (notably interactions between zero-valued expectations and short calldata, and the lack of ABI-decoding awareness for dynamic arguments) also warrant attention from anyone authoring allowlist rules, and a role-separation gap allows `MANAGER_ROLE` to substitute deposit-asset converters through the listing entrypoints that `DEFAULT_ADMIN_ROLE` is otherwise expected to gate.

The migration entrypoint concentrates the highest-impact risk identified during the audit: it hardcodes the post-migration price-per-share without consulting the underlying redenomination rate, accepts asset metadata without cross-checking the new underlying token, and provides no idempotency guard, so the operational assumption that it is invoked exactly once and immediately succeeded by an upgrade to the standard implementation is load-bearing. NAV-driven accounting more broadly delegates correctness to the off-chain price setter and the rate-limiter configuration; neither the posted price nor the consortium-published stakedLBTC ratio is constrained on chain to refresh within a bounded window, and the reconciliation logic introduced in the later revision of `postPricePerShare` still credits fee-share mints with imaginary backing because those mints carry no corresponding asset inflow. These trust assumptions are central to the security properties of the deposit and redemption

paths and should be reinforced operationally, with particular care taken to strengthen test coverage and documentation around the migration sequence before deployment.

OpenZeppelin would like to thank the Lombard team for their cooperation and responsiveness throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.